

Tim's ASCII Table for Crestron Programmers

A compact ASCII reference aimed at Crestron programming, serial/TCP protocols, SIMPL+, C#, and figuring out exactly what bytes a device is actually expecting.

This document lives at <https://github.com/CTI-Tim/CrestronCSharpExamples>

Full 7-bit ASCII Table

Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char
00	0	NUL	20	32	SPACE	40	64	@	60	96	`
01	1	SOH	21	33	!	41	65	A	61	97	a
02	2	STX	22	34	"	42	66	B	62	98	b
03	3	ETX	23	35	#	43	67	C	63	99	c
04	4	EOT	24	36	\$	44	68	D	64	100	d
05	5	ENQ	25	37	%	45	69	E	65	101	e
06	6	ACK	26	38	&	46	70	F	66	102	f
07	7	BEL	27	39	'	47	71	G	67	103	g
08	8	BS	28	40	(	48	72	H	68	104	h
09	9	HT	29	41	)	49	73	I	69	105	i
0A	10	LF	2A	42	*	4A	74	J	6A	106	j
0B	11	VT	2B	43	+	4B	75	K	6B	107	k
0C	12	FF	2C	44	,	4C	76	L	6C	108	l
0D	13	CR	2D	45	-	4D	77	M	6D	109	m
0E	14	SO	2E	46	.	4E	78	N	6E	110	n
0F	15	SI	2F	47	/	4F	79	O	6F	111	o
10	16	DLE	30	48	0	50	80	P	70	112	p
11	17	DC1	31	49	1	51	81	Q	71	113	q
12	18	DC2	32	50	2	52	82	R	72	114	r
13	19	DC3	33	51	3	53	83	S	73	115	s
14	20	DC4	34	52	4	54	84	T	74	116	t
15	21	NAK	35	53	5	55	85	U	75	117	u
16	22	SYN	36	54	6	56	86	V	76	118	v
17	23	ETB	37	55	7	57	87	W	77	119	w
18	24	CAN	38	56	8	58	88	X	78	120	x

Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char
19	25	EM	39	57	9	59	89	Y	79	121	y
1A	26	SUB	3A	58	:	5A	90	Z	7A	122	z
1B	27	ESC	3B	59	;	5B	91	[	7B	123	{
1C	28	FS	3C	60	<	5C	92	**	7C	124	
1D	29	GS	3D	61	=	5D	93	]	7D	125	}
1E	30	RS	3E	62	>	5E	94	^	7E	126	~
1F	31	US	3F	63	?	5F	95	_	7F	127	DEL

Common Control Characters

Name	Hex	Dec	Escape	Description
NUL	00	0	\0	Null
BEL	07	7	\a	Bell / alert
BS	08	8	\b	Backspace
HT	09	9	\t	Horizontal tab
LF	0A	10	\n	Line feed
VT	0B	11	\v	Vertical tab
FF	0C	12	\f	Form feed
CR	0D	13	\r	Carriage return
ESC	1B	27		Escape
DEL	7F	127		Delete

Crestron SIMPL+ Warning: \n Is Different

If you are coming from C, C++, C#, Python, or most other programming languages, this one can bite you.

In **Crestron SIMPL+**, \n does **not** mean only Line Feed (0A).

SIMPL+ treats:

```
\n
```

as:

```
Carriage Return + Line Feed
0D 0A
```

So in SIMPL+:

```
\n
```

is equivalent to:

```
\x0D\x0A
```

This matters when working with serial and TCP/IP protocols. If a device specifically requires only `LF` (`0A`), using `\n` in SIMPL+ will send an extra `CR` (`0D`) before it.

When the protocol specifies exact byte values, it is usually safer and clearer in SIMPL+ to write the bytes explicitly:

<code>\x0D</code>	<code>CR</code>
<code>\x0A</code>	<code>LF</code>
<code>\x0D\x0A</code>	<code>CR + LF</code>

Do not assume SIMPL+ escape sequences behave exactly like C# escape sequences just because they look familiar.

Code Pages: Where SIMPL+ and C# Stop Agreeing

This is an important one when moving protocol code between SIMPL/SIMPL+ and C#.

SIMPL and SIMPL+ are happiest in the ASCII world. Characters `00-7F` behave like normal ASCII characters. Once you get above `7F`, it is generally better to stop thinking of the value as a character and start thinking of it as **raw hex data in the string**.

That model works surprisingly well for a lot of Crestron protocol programming. A value such as:

```
\xFF
```

is simply the byte `FF`. You are not asking SIMPL+ what character `FF` represents. You want that byte sent.

C# strings are different. A C# `string` is Unicode text, and converting that text into bytes requires an **encoding**. Picking the wrong encoding can quietly change the data you put on the wire.

Why `Encoding.ASCII` Can Fail

It sounds like the obvious choice when talking to an ASCII-style device:

```
Encoding.ASCII.GetBytes(data);
```

That is fine as long as every value is inside the real 7-bit ASCII range:

```
00-7F
```

The problem starts when the protocol uses byte values from:

```
80-FF
```

Those values are **not ASCII**.

Encoding ASCII cannot represent them as equivalent single-byte characters. If you try to push Unicode characters outside its supported range through the ASCII encoder, they can be replaced rather than producing the byte value you expected.

For a binary-ish protocol where you thought you were sending:

```
02 FF 80 41 0D
```

that is a very bad place for a helpful text encoder to start making decisions for you.

Why Defaulting to UTF-8 Can Bite You Even Harder

UTF-8 is the normal modern choice for actual text, and for text it is absolutely the right answer most of the time.

Protocol bytes are not always text.

The first 128 UTF-8 values, 00-7F, happen to line up perfectly with ASCII. That makes everything look fine during testing until the protocol produces a value above 7F.

In UTF-8, many characters above that range require **multiple bytes**.

For example, the Unicode character `ÿ` is:

```
Unicode: U+00FF
```

Encoded as Latin-1:

```
FF
```

Encoded as UTF-8:

```
C3 BF
```

So code that appears to contain one character can suddenly put **two bytes on the wire**.

If the receiving device expected:

```
FF
```

and C# sends:

```
C3 BF
```

the protocol is now broken, and the code can look perfectly reasonable while you are staring at it wondering what went wrong.

Latin-1 Is Often the Safest String Encoding for 00-FF Protocol Work

When a Crestron protocol is being represented as a string and you need a predictable one-to-one mapping across the entire byte range, Latin-1 is usually the safest encoding to use in C#:

```
Encoding.Latin1
```

Latin-1 maps:

```
00-FF
```

directly to:

```
00-FF
```

One character value becomes one byte.

Character value	Latin-1 byte
00	00
41	41
7F	7F
80	80
FF	FF

There is no UTF-8 expansion and no ASCII replacement of values above `7F`.

For example:

```
byte[] bytes = Encoding.Latin1.GetBytes(data);
```

and going the other direction:

```
string data = Encoding.Latin1.GetString(bytes);
```

This gives C# behavior that is much closer to the way a Crestron programmer is accustomed to treating a SIMPL+ string containing byte values.

The Rule I Use

For normal human-readable text:

```
UTF-8
```

For a protocol that is guaranteed to contain only real 7-bit ASCII:

```
ASCII
```

For a Crestron-style protocol being carried in a C# string where every possible byte from 00-FF needs to survive unchanged:

```
Latin-1
```

And if the data is genuinely binary rather than text, the better answer in C# is usually to stop pretending it is a string at all and work with:

```
byte[]
```

The important part is to make the decision deliberately. Do not let the default encoding decide what bytes your protocol sends.

Common Line Terminators

Terminator	Hex	Typical C# Escape
CR	0D	\r
LF	0A	\n
CR LF	0D 0A	\r\n

SIMPL+ PROGRAMMERS: \n IS NOT LF ONLY. In SIMPL+, \n sends CR + LF (0D 0A). This is different from C# and many other languages, where \n means LF (0A) only. This bites programmers a LOT when moving protocol code between SIMPL+ and C#.

When a serial or TCP device wants a command terminator, check the protocol carefully. CR, LF, and CR LF are not interchangeable. If the protocol specifies exact bytes, especially in SIMPL+, use the explicit hex values when there is any doubt:

```
\x0D      CR
\x0A      LF
\x0D\x0A  CR + LF
```

Why Programmers Should Usually Use Hex

ASCII tables often show decimal because it is familiar to humans, but when working with protocols, byte streams, serial data, network captures, embedded systems, or debugging raw data, **hex is usually the better representation.**

A byte is 8 bits. In hex, one byte is always exactly two digits:

```
00
0D
20
41
7F
FF
```

That makes byte boundaries immediately obvious. Decimal does not have that property:

```
0
13
32
65
127
255
```

Those values are the same numbers, but the decimal representation changes width and hides the structure of the byte.

Hex also lines up naturally with binary because each hex digit represents exactly four bits:

```
Hex:      4    1
Binary: 0100 0001
```

So ASCII `A` is `0x41`. Looking at `41`, it is easy to split the byte into two nibbles, `4` and `1`. Decimal `65` represents the same value, but tells you almost nothing about the underlying bit pattern.

This becomes even more useful when reading protocol documentation or packet captures:

```
48 45 4C 4C 4F 0D 0A
H  E  L  L  O  CR  LF
```

Compare that with decimal:

```
72 69 76 76 79 13 10
```

It works, but it is harder to scan, harder to recognize boundaries, and harder to compare against the actual bytes being transmitted.

Use decimal when the number itself matters to a person. Use hex when the bits and bytes matter to the programmer.

Why the ASCII Alphabet Is Easy to Recognize in Hex

One of the nice things about ASCII is that the alphabet lines up very cleanly in hex. Uppercase and lowercase letters have the same low five bits, and the lowercase version is exactly `0x20` higher than the uppercase version.

Letter	Upper Hex	Lower Hex	Letter	Upper Hex	Lower Hex
A / a	41	61	N / n	4E	6E
B / b	42	62	O / o	4F	6F
C / c	43	63	P / p	50	70
D / d	44	64	Q / q	51	71

Letter	Upper Hex	Lower Hex	Letter	Upper Hex	Lower Hex
E / e	45	65	R / r	52	72
F / f	46	66	S / s	53	73
G / g	47	67	T / t	54	74
H / h	48	68	U / u	55	75
I / i	49	69	V / v	56	76
J / j	4A	6A	W / w	57	77
K / k	4B	6B	X / x	58	78
L / l	4C	6C	Y / y	59	79
M / m	4D	6D	Z / z	5A	7A

Put the full ranges beside each other and the pattern becomes obvious:

Upper:	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Hex:	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	50	51	52	53	54	55	56	57	58	59	5A
Lower:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Hex:	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	70	71	72	73	74	75	76	77	78	79	7A

That 0x20 relationship is not an accident:

A = 0x41	a = 0x61
B = 0x42	b = 0x62
C = 0x43	c = 0x63
0x61 - 0x41 = 0x20	

Once a programmer gets used to looking at ASCII in hex, values such as 41, 4D, 61, 6D, 30, 0D, and 0A start becoming recognizable without needing to look them up.

Control Character Names

Hex	Dec	Name	Meaning
00	0	NUL	Null
01	1	SOH	Start of Heading
02	2	STX	Start of Text
03	3	ETX	End of Text
04	4	EOT	End of Transmission

Hex	Dec	Name	Meaning
05	5	ENQ	Enquiry
06	6	ACK	Acknowledge
07	7	BEL	Bell
08	8	BS	Backspace
09	9	HT	Horizontal Tab
0A	10	LF	Line Feed
0B	11	VT	Vertical Tab
0C	12	FF	Form Feed
0D	13	CR	Carriage Return
0E	14	SO	Shift Out
0F	15	SI	Shift In
10	16	DLE	Data Link Escape
11	17	DC1	Device Control 1
12	18	DC2	Device Control 2
13	19	DC3	Device Control 3
14	20	DC4	Device Control 4
15	21	NAK	Negative Acknowledge
16	22	SYN	Synchronous Idle
17	23	ETB	End of Transmission Block
18	24	CAN	Cancel
19	25	EM	End of Medium
1A	26	SUB	Substitute
1B	27	ESC	Escape
1C	28	FS	File Separator
1D	29	GS	Group Separator
1E	30	RS	Record Separator
1F	31	US	Unit Separator

Hex	Dec	Name	Meaning
7F	127	DEL	Delete

Quick Hex / Decimal Notes

- ASCII is a 7-bit character set, so the standard range is decimal 0-127 or hex 00-7F.
- Hex values 30-39 are the characters 0-9.
- Hex values 41-5A are uppercase A-Z.
- Hex values 61-7A are lowercase a-z.
- The difference between an uppercase and lowercase ASCII letter is hex 20 or decimal 32.

A few useful examples

Character	Hex	Decimal
0	30	48
9	39	57
A	41	65
Z	5A	90
a	61	97
z	7A	122
SPACE	20	32

Compact ASCII reference intended as a quick programming and protocol-debugging reference.